

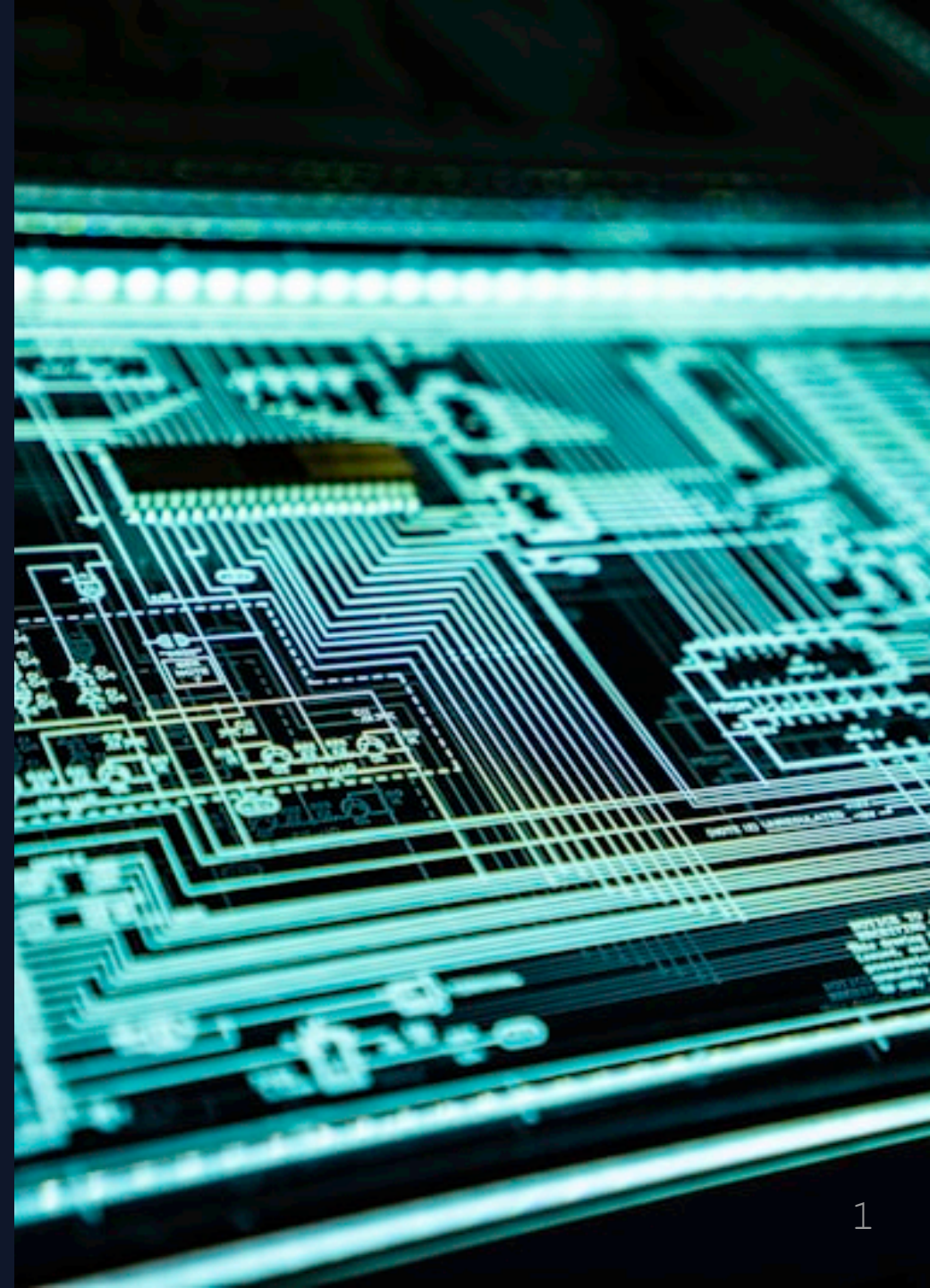


L15: Loops in Graphics

Pattern Recognition & Efficiency

Neon City Cyber Academy

Module 4: Visual Algorithms





Your Team

Emily Chen - *Handler*

"Programmers are lazy - in a good way. Never write the same code twice when you can loop it."

Cole Martinez - *Tech Lead*

"A square is 4 lines. A hexagon is 6 lines. Writing each one manually? Amateur hour. Loop it."



The Problem: Repetitive Code

Bad Code (Beginner) :

```
# Draw square manually  
print("Move 100")  
print("Turn 90")  
print("Move 100")  
print("Turn 90")  
print("Move 100")  
print("Turn 90")  
print("Move 100")  
print("Turn 90")
```

Lines of code: 8

Maintainability: Terrible

Efficiency: 0/10

✓ The Solution: Loops

Good Code (Professional):

```
# Draw square with loop
for i in range(4):
    print("Move 100")
    print("Turn 90")
```

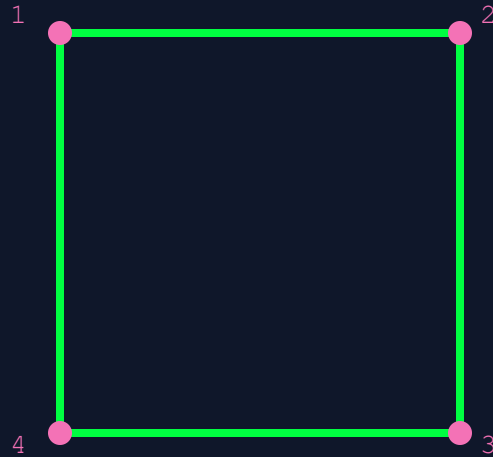
Lines of code: 3

Maintainability: Excellent

Efficiency: 10/10

THIS is why loops exist!

Square Pattern (Visual)



`4 sides = range(4)`

Emily's Loop Logic:

"Ask yourself: What repeats?"

Square: 4 identical sides

Hexagon: 6 identical sides

Circle (approximation): 360 tiny segments

"The pattern: `for i in range(N)` where N = number of repetitions"

◆ Hexagon Pattern

6 sides = range(6)

Each turn: $360/6 = 60^\circ$

12 34 Calculating Total Distance

```
# Square perimeter
side_length = 100
num_sides = 4

total_distance = side_length * num_sides
print(f"Total distance: {total_distance}") # 400
```

This is text output - perfect for Judge0!

Cole's Efficiency Rule:

"If you're copy-pasting code, you're doing it wrong."

```
# ❌ BAD: Hard to modify
print("Forward 50")
print("Forward 50")
print("Forward 50")

# ✅ GOOD: Change once, affects all
for i in range(3):
    print("Forward 50")
```

"Want to change 50 to 100? One edit vs three edits."

Pattern: Triangle

3 sides = range(3)

Each turn: $360/3 = 120^\circ$



The General Formula

For ANY regular polygon:

```
num_sides = 8 # Octagon
turn_angle = 360 / num_sides # 45 degrees

for i in range(num_sides):
    print(f"Move 100")
    print(f"Turn {turn_angle}")
```

Works for triangle (3), square (4), pentagon (5), hexagon (6)...



Distance Tracking Example

```
side_length = 50
num_sides = 6 # Hexagon

# Calculate total distance BEFORE drawing
total = 0
for i in range(num_sides):
    total += side_length

print(f"Drone will travel {total} units")
# Output: Drone will travel 300 units
```



Emily's Real-World Example:

"Game development uses this constantly:"

```
# Spawn 10 enemies in a circle
num_enemies = 10
for i in range(num_enemies):
    angle = (360 / num_enemies) * i
    print(f"Spawn enemy at angle {angle}")
```

Output:

```
Spawn enemy at angle 0.0
Spawn enemy at angle 36.0
Spawn enemy at angle 72.0
...
```



Nested Loops: Grid Pattern

```
# Create 3x3 grid
for row in range(3):
    for col in range(3):
        x = col * 100
        y = row * 100
        print(f"Place marker at ({x}, {y})")
```



12
34

Challenge: Star Pattern

```
# 5-pointed star
num_points = 5
for i in range(num_points):
    print("Move 100")
    print("Turn 144") # 720/5 = 144 degrees

print("Star complete!")
```

Why 144? Stars use 720° instead of 360° !

✓ Lesson Complete!

You Mastered:

- ✓ Loops for repetitive patterns
- ✓ `range(N)` for N repetitions
- ✓ Distance calculation with loops
- ✓ Turn angles: $360/\text{sides}$
- ✓ Nested loops for grids
- ✓ Code efficiency principles

Next: L16 - Conditional Drawing

Boundary checking and collision detection



Your Assignment

Create a **Pattern Generator**:

1. Write a function `polygon(sides)` that prints commands for ANY polygon
2. Calculate turn angle: $360 / \text{sides}$
3. Use a loop to print move/turn commands
4. Test with triangle (3), square (4), hexagon (6)